Trabajo de Taller IISS 2026 — Iteración 5

Calidad, tiempo acelerado y cobertura

Objetivo

En esta iteración se busca lograr características de calidad:

- ¿Cómo responde el controlador a distintas situaciones iniciales?
- ¿Qué acciones realiza sobre la calefacción al iniciar / terminar su funcionamiento?
- ¿Qué ocurre cuando un termostato deja de funcionar?
- ¿Cómo reacciona cuando se cambia la configuración de habitaciones o temperatura en el medio de la ejecución?
- ¿Qué pasa cuando un switch no responde?
- ¿Qué sucede cuando el MQTT server cae y luego se recupera (endurecimiento del diseño previsto en la Iteración 4).

Además, para la prueba final del prototipo, el simulador incorpora un mecanismo de tiempo acelerado que debe ser soportado.

Se va a recibir una nueva versión del simulador que va a incluir, al inicio, un parámetro con el factor de aceleración que debe ser tenido en cuenta por el controlador.

El equipo debe adoptar un conjunto de estándares de codificación y asegurar que el código responda a los mismos. Se debe entregar un reporte de cobertura de los tests, a nivel del core, no inferior al 85 %.

Entregables de la iteración

- Componente desarrollado y con las **pruebas unitarias verificadas**; pruebas actualizadas.
- **Soporte de tiempo acelerado**: el controlador respeta el factor de aceleración que el simulador envía al inicio.
- **Estándares de codificación** adoptados, con evidencia de que el código los cumple.
- **Reporte de cobertura** de las pruebas, a nivel del **core no inferior al 85 %**.
- **Tests iniciales de integración**, al menos tres casos:
  - control de temperatura cuando la carga total no es una limitación;
  - control cuando la carga total es una limitación efectiva;
  - casos borde: caída y recuperación del MQTT server, pérdida de mensajes, falla en la comunicación REST.

- Verificación del comportamiento ante distintas situaciones iniciales y en el arranque / parada del controlador.
- **Diagramas 4+1:** se completan/actualizan vistas (avanzando hacia al menos un diagrama por vista).

Mecanismo de entrega

- Tag `version5` en GitHub. Copia `.zip` en Moodle.
- Archivos clave:
  - `modulos … (fuentes de los componentes).`
  - tests de integración (junto al código)
  - `/docs/calidad/estandares-codificacion.md`
  - `/docs/arquitectura/4mas1.md` (actualizado)
  - reporte de **cobertura** del core (≥ 85 %), p. ej. `/docs/calidad/cobertura/…`
  - `/docker/docker-compose.yml`
  - `/scripts/build.sh`
  - `/README.md` (actualizado)

Característica opcional — API del Controlador por MCP

> **Opcional.** No forma parte de los entregables obligatorios de esta iteración. Se valora como característica diferencial del equipo.

Entre los elementos considerados para el producto está la posibilidad de **dialogar con el sistema** mediante GenAI. Esta característica lo concreta: exponer las operaciones del Controlador como un **servidor MCP** (Model Context Protocol), de modo que un cliente compatible pueda configurar y operar el sitio de forma conversacional.

El objetivo de ingeniería es **poner a prueba la modularidad** lograda con la separación core/engine: si el diseño es correcto, ofrecer una segunda vía de acceso a las mismas operaciones debe costar poco y no debe requerir tocar la lógica de control.

Desarrollar un **servidor MCP mínimo**, usando **Spring AI MCP** (coherente con el Spring Boot ya adoptado), que exponga como **herramientas** (*tools*) las operaciones que el sistema ya ofrece.

> **Referencia:** Spring AI — *Getting Started with MCP*: https://docs.spring.io/spring-ai/reference/guides/getting-started-mcp.html (versión actual: **2.0.0**). El punto de partida es el starter `spring-ai-starter-mcp-server-webmvc`, con transporte **Streamable HTTP** —el que corresponde para desplegar el servidor como un servicio más del `docker-compose`.

Operaciones a exponer:

- **Configurar dispositivos:** consultar y modificar la definición de las habitaciones (termostato, switch, temperatura esperada).
- **Activar o desactivar:** accionar los switches (ON / OFF).

- **Conocer el estado actual:** temperatura y estado de cada habitación, y estado del Controlador (en marcha o detenido).

Restricciones:

- El servidor MCP es un **adaptador delgado** sobre las operaciones ya existentes: no incorpora lógica de control ni persistencia propias.
- Se despliega como un contenedor más del `docker-compose`.
- Cada herramienta se define con su **nombre, descripción y esquema de argumentos**. La descripción es parte del contrato: está escrita para que un modelo decida cuándo usarla. El diseño concreto de las herramientas queda a cargo del equipo.

Entregable adicional (se entrega con el mismo tag `version5`):

- **Servidor MCP** operativo, integrado al `docker-compose`.
- **Definición de las herramientas** expuestas (nombre, descripción y esquema).
- **Demostración** de las tres operaciones (configurar, activar/desactivar, consultar estado) desde un cliente MCP o una herramienta de inspección.
- **Documento breve** comparando el diseño del **API REST** (recursos y verbos) con el del **API MCP** (herramientas y descripciones), señalando qué cambia cuando el consumidor del contrato es un modelo y no un programa.

Archivos clave: `/modulos/mcp/`… (servidor MCP, como módulo Maven), `/docs/mcp/herramientas.md`, `/docs/mcp/rest-vs-mcp.md`, y el `docker-compose.yml` actualizado.
